

Decision-OS V11: Forget for Future

Reconnectable Forgetting for Long-Horizon Agentic AI

Shinichi Nagata

Abstract

Long-horizon agentic AI systems increasingly operate across conversations, files, tool calls, code changes, evidence anchors, unresolved assumptions, and handoffs between sessions or models. In such systems, memory is both a capability and a burden. Short-term performance may benefit from retaining more history, but long-term operation can become dominated by rereading, retrieval, reinterpretation, obsolete assumptions, and accumulated self-reference.

This paper introduces *Reconnectable Forgetting* as a memory-economy principle for long-horizon agentic AI. Forgetting is not treated as deletion, loss, or ordinary summarization. It is defined as a controlled compression operation that preserves the structures required for future re-entry: evidence anchors, temporal coordinates, stop conditions, unresolved deltas, provenance, and re-entry paths.

The central claim is that self-evolving systems do not require perfect memory. They require the ability to forget in a way that remains reconnectable. A deeper failure arises when a system repeatedly consumes its own prior outputs as the next premise within a single continuous active context. This paper refers to that failure as *self-referential premise corrosion*. Each local step may appear coherent, while the original assumptions, constraints, unknowns, stop conditions, and judgment boundaries gradually drift.

To address this problem, the paper defines a three-layer memory architecture: the *Active Context Layer*, the *Compressed Residue Layer*, and the *Provenance-Key Layer*. It then introduces *Decision-Equivalent Compression*, *Judgment Fidelity*, and a *Reconnectability Gate* with PASS/DELAY/BLOCK outputs. The paper also defines bounded controls for recompression stopping, evaluation budgeting, and a Discovery Lane for low-impact residues that may become future discovery candidates.

The framework distinguishes origin identity from judgment fidelity. Provenance can show where a compressed memory came from, but it does not by itself prove that the compressed memory preserves the judgment-critical structure required for future reuse. Accordingly, compressed memory must be evaluated before it is reused as a judgment ground.

This paper should be read as a conceptual and operational framework paper. It does not claim to implement a complete memory system, a cryptographic zero-knowledge protocol, or an empirical validation of all proposed gates. Its contribution is to define the minimal conditions under which forgetting can reduce context burden without destroying future re-entry. The goal of V11 is not total recall. Its goal is to allow memory to become light enough to continue, while remaining structured enough for the future self to reconnect.

Index Terms

Reconnectable Forgetting, Forget for Future, memory economy, long-horizon agentic AI, self-referential premise corrosion, Provenance-Key Layer, Decision-Equivalent Compression, Judgment Fidelity, Reconnectability Gate, Discovery Lane, line self-recursion

I. INTRODUCTION

Memory is both a capability and a burden.

A short-lived AI system can often operate from the current prompt alone. A long-horizon agent cannot. It accumulates conversations, files, tool calls, code changes, evidence anchors, failed attempts, revisions, unresolved assumptions, and human feedback. At first, this accumulated history appears to strengthen the system. It provides continuity and gives future decisions more context.

However, as the horizon becomes longer, memory can begin to work against the agent. The system must reread, retrieve, reinterpret, and reconnect its own past before it can act in the present. Old assumptions may re-enter new judgments. Expired constraints may remain active. Unknowns may be smoothed into conclusions. Stop conditions may weaken into ordinary notes.

The intuitive solution is to remember more: keep the full conversation, preserve every file, and reload the entire history whenever continuation is needed. Existing agent-memory work also recognizes the need to manage limited context through memory hierarchies or virtual context mechanisms [1]. In long-horizon agentic systems, however, the problem is not only whether history can be stored or retrieved. The problem is whether a future agent can reconnect to that history under the right time, scope, evidence, and capability conditions.

This paper therefore begins from a different claim:

Self-evolving AI systems do not require perfect memory. They require forgetting that remains reconnectable.

V11 calls this principle *Reconnectable Forgetting*. Forgetting is not treated as deletion, ordinary summarization, or passive loss caused by context limits. It is defined as a controlled compression operation that preserves the structures required for future re-entry: evidence anchors, temporal coordinates, stop conditions, unresolved deltas, provenance, and re-entry paths.

The deeper failure addressed by V11 is not memory shortage alone. It is self-reference under compression. When an agent repeatedly consumes its own prior outputs as the next premise inside a single continuous active context, each local step may appear coherent while the original assumptions, constraints, unknowns, stop conditions, and judgment boundaries gradually drift. This paper refers to that failure as *self-referential premise corrosion*.

V11 does not claim to solve every self-reference problem. Its operational response is narrower: it breaks the assumption that long-horizon continuity must remain inside one uninterrupted active context. Instead, it treats continuity as a line maintained through compressed residues, evidence anchors, As-of conditions, stop conditions, deltas, and re-entry paths.

A point may close. A context may end. A model may change. The line can continue if the future evaluator can reconnect.

The central question of this paper is therefore:

How can a long-horizon agent forget past context without losing the ability to reconnect, verify, re-evaluate, and reuse what matters?

This paper contributes four components.

First, it defines Reconnectable Forgetting as controlled compression with a path back. Second, it introduces a three-layer memory architecture: Active Context, Compressed Residue, and Provenance-Key. Third, it defines Decision-Equivalent Compression and Judgment Fidelity as criteria for preserving judgment-critical structure rather than merely semantic meaning. Fourth, it introduces a Reconnectability Gate with PASS, DELAY, and BLOCK outputs.

The contribution is not a complete implementation of long-term memory. It is a conceptual and operational framework for deciding what should remain active, what should be compressed, what should be anchored, what should be protected, and what may be closed.

In the Decision-OS lineage, V10 addressed survival-bounded planning: how an agent preserves aspiration by rescaling the trajectory when the original goal-length becomes non-survivable. V11 addresses the next memory-economy question: what the surviving agent can stop carrying at full resolution without losing future re-entry. V12 then addresses closure: how an update can be completed in a form that the future self can restart.

V11 is therefore positioned between survival and completion. It asks how memory becomes light enough for the system to continue, while remaining structured enough for the future self to return.

The thesis can be stated in one line:

Future requires forgetting, but forgetting must remain reconnectable.

II. PROBLEM: SINGLE-CONTEXT RECURSION AND PREMISE CORROSION

The central problem of V11 is not that long-horizon agents remember too little. The problem is that they often remember in a form that becomes difficult to reuse safely.

A short interaction can be handled as a point. The system receives an input, produces an output, and closes the task. A long-horizon agent does not operate as a point. It operates across prior conversations, files, decisions, tool calls, code changes, corrections, unresolved assumptions, and human feedback. As this history grows, the agent faces a structural tension: too little memory destroys continuity; too much active memory overloads the present; ordinary summaries may preserve fluent meaning while destroying judgment-critical structure.

The visible symptom is memory growth. The deeper failure is self-reference.

A long-running agent may repeatedly consume its own prior outputs as the premises of later reasoning. A prior answer was already a compression of an earlier context. If that compressed answer becomes the basis for another answer, and this process repeats, the system may gradually lose access to the original evidence, assumptions, constraints, unknowns, stop conditions, and scope boundaries.

V11 calls this failure *self-referential premise corrosion*.

Premise corrosion is dangerous because it does not necessarily appear as a hallucination or contradiction. Each local step may sound coherent. The system may preserve the topic, tone, and apparent meaning of the prior context while losing the conditions that made the prior judgment valid. In V11 terms, meaning may remain while judgment fidelity is lost.

Typical signs include:

- assumptions becoming implicit rather than declared;
- unknowns being converted into apparent conclusions;

- stop conditions weakening into ordinary cautions;
- evidence anchors being replaced by narrative confidence;
- local summaries being treated as original context;
- prior decisions being reused outside their original scope.

Total memory does not solve this problem. Preserving every conversation, file, tool call, correction, and evidence anchor appears safe because nothing is intentionally lost. However, full-resolution memory increases rereading cost, retrieval cost, latency, and attention pressure. It also does not automatically preserve scope, judgment boundaries, capability ceilings, or valid reuse conditions. Raw history can still be misused if the future agent does not know how to re-enter it.

Ordinary summarization also does not solve the problem. Summaries optimize for readability and semantic compression, not for future judgment. A summary may accurately state that a decision was made carefully while omitting the evidence anchor, unknown set, stop condition, capability ceiling, or recheck trigger. In that case, the summary preserves the impression of reasoning while destroying the structure required to reuse that reasoning.

The unstable assumption behind both failures is single-context recursion: the belief that continuity should be maintained by keeping one active context alive. In practice, long-horizon systems cross boundaries. Sessions end, models change, files move, tools update, prompts are rewritten, users revise goals, and future agents operate under different capability ceilings.

If continuity depends on one uninterrupted active context, every boundary becomes a failure. V11 instead asks whether continuity can survive context boundaries.

The problem addressed by V11 can therefore be stated as follows:

Long-horizon agentic AI cannot safely rely on either total memory or ordinary summarization. It requires a memory economy that reduces active context burden while preserving future re-entry into judgment-critical structure.

This produces the minimal requirement of the paper:

A long-horizon agent may forget only if the future self can reconnect to the evidence, scope, conditions, unresolved deltas, and stop rules required to evaluate what was forgotten.

If those elements are preserved, forgetting becomes a controlled memory operation. If they are lost, the system has not forgotten safely. It has merely lost the past in a more fluent form.

III. RECONNECTABLE FORGETTING

Reconnectable Forgetting is the central principle of V11.

It is not deletion, ordinary summarization, total memory, or passive loss caused by context limits. It is a controlled compression operation that reduces active context burden while preserving the structures required for future re-entry.

In minimal form:

Reconnectable Forgetting means forgetting with a path back.

Let a raw memory item or history segment be denoted by x . Let $c(x)$ denote its compressed residue. The purpose of $c(x)$ is not to reproduce every expression, token, or local detail of x . Its purpose is to preserve the structures that allow a future evaluator to reopen, verify, revise, or stop the memory when needed.

A memory item is reconnectably forgotten when its compressed form preserves at least the following:

- the relevant claim, decision, or unresolved residue;
- the evidence anchor or source reference;
- the As-of condition under which it was formed;
- the scope and objective of the original judgment;
- the stop, recheck, or invalidation condition;
- the open delta or unresolved assumption;
- the re-entry path by which the future evaluator can reopen it.

This definition shifts forgetting from loss to controlled handoff. The raw history may leave active context, but the future system retains the ability to reconnect to what matters.

Deletion breaks the path back. If a memory is deleted without residue, provenance, evidence, or re-entry condition, the future agent cannot determine what was lost, why it mattered, or when it should be revisited. Deletion may be necessary for privacy, legal, safety, or operational reasons, but it is not Reconnectable Forgetting.

Ordinary summarization is also insufficient. A summary may preserve topic, readability, and semantic meaning while losing the judgment-critical structure required for reuse. For example, a summary may say that a decision was made cautiously, while removing the evidence anchor, unknown set, stop condition, capability ceiling, or recheck trigger. Such a summary preserves narrative meaning but destroys reconnectability.

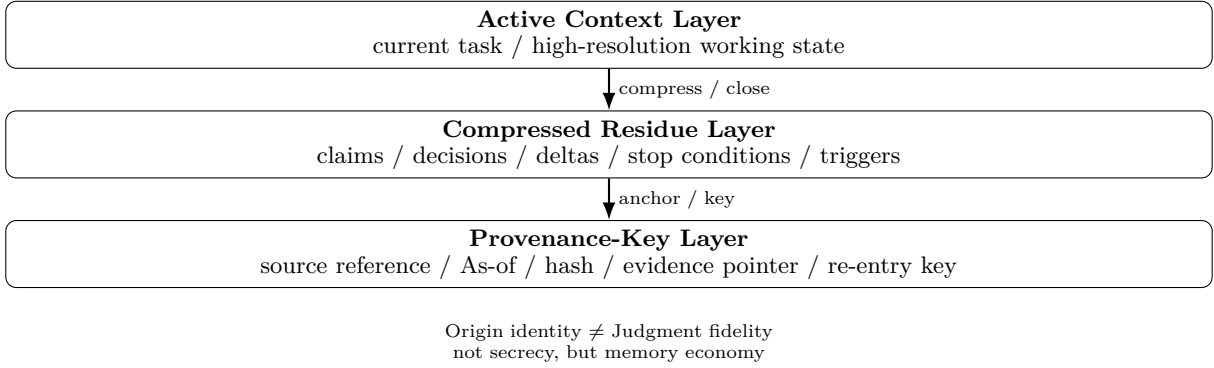


Fig. 1. Minimal memory architecture for Reconnectable Forgetting. Active context supports current high-resolution work, compressed residue preserves reusable judgment structure, and the Provenance-Key Layer preserves origin identity and re-entry keys without expanding raw history back into active context.

Total memory fails in the opposite direction. Keeping everything active appears safe because nothing is intentionally lost, but long-horizon systems cannot treat active context as an infinite archive. Full-resolution memory increases rereading cost, retrieval cost, attention pressure, and the risk that obsolete assumptions re-enter current judgment. The goal is not to keep every detail active. The goal is to preserve future access to the structures that can change judgment.

V11 therefore occupies a middle position:

not total recall, not ordinary summary, but reconnectable compression.

The term *future self* is operational. It does not require the same model, session, prompt, or active context. The future evaluator may be a later session of the same agent, a different model continuing the work, a human operator returning to the project, or an audit process evaluating a prior decision.

The relevant condition is not substrate identity. The relevant condition is whether the future evaluator can reconnect to the past judgment environment.

This is why V11 treats continuity as a line rather than a point. A point can close. A context can end. A model can change. Continuity remains possible if the line preserves re-entry.

The minimal rule is:

A memory may leave active context only if the future system retains enough structure to reconnect to its evidence, scope, time, stop condition, unresolved delta, and re-entry path.

If this condition is met, forgetting becomes a memory-economy operation. If it is not met, forgetting becomes loss.

IV. MINIMAL MEMORY ARCHITECTURE

Reconnectable Forgetting requires more than a shorter memory. It requires an architecture that separates what must remain active, what can be compressed, and what must remain anchored for future re-entry.

This separation follows the same structural discipline as V6: just as V6 separates candidate frontier from committed canonical state and applies conservative checks at the commitment boundary, V11 separates active context from compressed residue and applies judgment-fidelity checks before compressed memory is reused as judgment material.

V11 therefore defines a minimal three-layer memory architecture:

- 1) *Active Context Layer*
- 2) *Compressed Residue Layer*
- 3) *Provenance-Key Layer*

These layers are not implementation-specific modules. They are operational roles. A practical system may implement them through files, vector stores, structured logs, hashes, retrieval tools, human notes, or other memory mechanisms. The framework does not depend on a specific storage technology.

Figure 1 summarizes the three-layer structure. The goal is not to move all memory into a single compressed store. The goal is to control memory resolution according to its role in future judgment.

A. *Active Context Layer*

The *Active Context Layer* contains the high-resolution material currently used for reasoning and action. It includes the present prompt, recent conversation, open files, current task constraints, unresolved local details, and immediate user intent.

This layer is powerful because it preserves detail. It allows the agent to work with nuance, wording, local constraints, and recent corrections.

However, it is not a stable long-term memory layer. If too much history remains active, the agent becomes overloaded. Old assumptions may influence present decisions. Expired constraints may remain visible. Prior local choices may be mistaken for durable commitments.

The Active Context Layer should therefore be treated as a workbench, not as an archive. It supports present work, but it should not be forced to carry the entire past. The handoff point is not fixed. It adapts to both context coherence and impact level. When the active context is polluted, repetitive, or no longer preserves judgment boundaries, the handoff window is narrow and compression should occur early. Under ordinary progress, the system may choose a convenient work boundary, such as a section, figure, review pass, or task milestone. When the context remains coherent and continues to increase judgment value, the active context may be extended longer. However, high-impact or irreversible material should be moved toward anchored residue earlier even when the loop appears healthy. Thus, V11 treats handoff timing as an adaptive window: failed contexts close early, normal contexts close at convenient progress boundaries, and high-value coherent contexts may close later, provided that future re-entry remains preserved.

B. Compressed Residue Layer

The *Compressed Residue Layer* contains past material that has been reduced into reusable structure.

A residue is not a full transcript. It is not merely a summary. It is a compact record of what may matter for future re-entry.

Typical residue elements include:

- claims and decisions;
- constraints and dependencies;
- unresolved deltas;
- failure modes;
- evidence pointers;
- stop conditions;
- recheck triggers;
- closure reasons;
- scope boundaries.

The Compressed Residue Layer reduces active burden while preserving decision-relevant structure. It allows the future agent to continue without reloading the entire past.

However, compressed residue does not automatically have judgment authority. A residue may be incomplete, too coarse, or detached from its original conditions. It must therefore be evaluated before it is reused as a judgment ground.

This is why V11 later introduces Decision-Equivalent Compression and the Reconnectability Gate.

C. Provenance-Key Layer

The *Provenance-Key Layer* preserves the origin and re-entry keys of compressed memory.

It answers questions such as:

- Where did this residue come from?
- When was it created?
- Which file, log, version, artifact, or source does it refer to?
- What evidence anchor supports it?
- How can the future agent reopen or verify it?

In this paper, Provenance-Key Compression refers to a pointer-preserving memory-economy principle: preserving provenance and re-entry keys without expanding raw content into active context. This use of provenance is consistent with the broader role of provenance models in representing the origin and history of data or artifacts [2]. It allows a system to keep a structured path back to the source while avoiding full raw-history expansion during ordinary reasoning. This is not a cryptographic claim and does not implement a zero-knowledge protocol.

The purpose is not secrecy. The purpose is memory economy.

The Provenance-Key Layer allows a system to avoid loading raw history into active context while still preserving a path back to the source.

D. Origin Identity Is Not Judgment Fidelity

The Provenance-Key Layer can support *origin identity*. It can show where a residue came from and how to reopen the source.

However, origin identity is not the same as judgment fidelity.

A compressed memory may have a correct source reference and still be unsafe for judgment. For example, the residue may point to the correct file while omitting the stop condition. It may preserve the correct timestamp while losing the capability ceiling. It may preserve the correct source while smoothing an unresolved assumption into a conclusion.

Therefore, provenance is necessary but not sufficient. It gives the future agent a path back. It does not prove that the compressed residue preserved the judgment-critical structure of the original memory.

V11 fixes this distinction:

Origin identity answers where the memory came from. Judgment fidelity asks whether the memory still preserves the structure needed for future judgment.

E. Resolution Control

The three-layer architecture also supports resolution control.

Not every memory should be compressed at the same level. Low-impact and reversible material may be stored as coarse residue. High-impact or irreversible material may require detailed residue, stronger evidence anchors, or protected archival storage. Critical structures may not be suitable for ordinary compression at all.

The required resolution depends on factors such as:

- impact;
- reversibility;
- evidence dependence;
- responsibility or seat;
- stop-condition sensitivity;
- future re-entry cost;
- risk of misuse if the memory is over-compressed.

The architecture therefore does not say that all memory should be compressed. It says that memory should be assigned to the appropriate layer and resolution.

F. Minimal Rule

The minimal rule of the V11 architecture is:

Active context is for present high-resolution work. Compressed residue is for future reusable structure. The Provenance-Key Layer is for origin identity and re-entry. None of these alone guarantees judgment fidelity.

This architecture allows the agent to reduce context burden without pretending that compression is automatically safe. It prepares the ground for the next question: when is a compressed memory equivalent enough to support future judgment?

V. DECISION-EQUIVALENT COMPRESSION

Reconnectable Forgetting requires compression. However, not every compression is safe. A compressed memory may be short, fluent, and semantically close to the original while still being unusable for future judgment.

V11 calls the required standard *Decision-Equivalent Compression*.

Let x denote an original memory item, history segment, or decision context. Let $c(x)$ denote its compressed representation. A compressed memory $c(x)$ is decision-equivalent to x when it preserves enough judgment-critical structure for a future evaluator to recover the same or safer decision state under the declared scope and As-of evidence.

Decision-equivalence does not require full reconstruction. It does not require the same wording, token sequence, or local narrative. It requires preservation of the structures that can change a future judgment.

In short:

Decision-Equivalent Compression does not preserve the answer alone. It preserves the conditions under which the answer can be evaluated.

This separates two standards:

- **Meaning preservation:** whether the compressed memory says something similar.
- **Judgment preservation:** whether the compressed memory preserves the structure needed to evaluate, reuse, revise, or stop the prior judgment.

A compression may preserve meaning while losing judgment. For example, the statement “the release was acceptable” may preserve a prior outcome. But if it removes the observation window, evidence anchor, unresolved risks, stop condition, or capability ceiling, the future evaluator cannot know whether the prior permission still applies.

To make judgment fidelity operational, V11 defines a set of declared judgment-critical queries Q_j . These are the questions that must remain answerable after compression for the memory to remain usable as judgment material. Typical queries include:

- What claim, decision, or residue was preserved?
- What evidence supported it?
- Under what As-of condition was it formed?
- What was the original scope?
- What assumptions or unknowns remained?
- What stop, recheck, or invalidation condition applied?
- Was the prior state PASS, DELAY, or BLOCK?
- What would make the future evaluator refuse reuse?

V11 defines *Judgment Fidelity* as the preservation of judgment-critical structure under compression. A compressed memory is judgment-faithful when, under the same As-of evidence and declared scope, it satisfies the following conditions:

- 1) It preserves the same answer to all declared judgment-critical queries in Q_j , or downshifts conservatively.
- 2) It preserves the same or safer PASS/DELAY/BLOCK outcome.
- 3) It preserves the evidence anchors required to replay or verify the judgment.
- 4) It preserves stop, recheck, and invalidation conditions.
- 5) It preserves the relevant As-of context and capability ceiling.
- 6) It does not convert unknowns or assumptions into settled conclusions.

A lightweight operational check is the *Outcome-Masked Replay Test*. In this test, the future evaluator hides the original outcome and attempts to reconstruct the gate-relevant state using only the compressed memory, its evidence anchor, and the declared judgment-critical queries Q_j . The test is passed when the evaluator recovers the same or safer PASS/DELAY/BLOCK state and can identify the evidence, scope, As-of condition, stop or recheck condition, and unresolved deltas that supported the original judgment.

A failed replay does not automatically constitute BLOCK; it is a signal to identify which condition was missing and whether it is repairable.

The criterion is intentionally conservative. A compressed memory may become less confident. It may downshift from PASS to DELAY. It may request re-anchoring. These are acceptable. What it must not do is upgrade uncertainty into permission.

The most dangerous compression failure is an unsafe upgrade. An unsafe upgrade occurs when compression makes a memory appear more reusable, more certain, or more complete than the original judgment allowed.

Examples include:

- DELAY becoming PASS without new evidence;
- BLOCK becoming DELAY or PASS through omission;
- an unresolved assumption becoming an accepted premise;
- a stop condition becoming a weak caution;
- an evidence-dependent claim becoming a general claim;
- an As-of judgment being reused outside its original time boundary.

V11 therefore fixes the following rule:

Compression may downshift uncertainty. It must not upgrade uncertainty into permission.

Decision-equivalent compression also requires evidence anchors. An evidence anchor is a path back to the source material that supported the original judgment. It may be a file, log, version, timestamp, hash, commit, artifact ID, citation, screenshot, or other reference. Without an evidence anchor, the compressed memory becomes a free-standing statement. It may be useful as a note, but it should not be reused as a judgment ground.

Finally, decision-equivalent compression must preserve difference recoverability. Compression always loses something. The requirement is not perfect recovery, but traceable difference. If a future evaluator disagrees with the compressed memory, the system should be able to identify whether the difference came from changed evidence, changed scope, omitted assumptions, shifted capability ceilings, lost stop conditions, or compression loss.

Recoverable difference is acceptable. Untraceable difference is not.

The minimal rule is:

A compressed memory is judgment-equivalent only if it preserves the judgment-critical structure required to recover the same or safer decision state.

If compression changes a judgment-critical answer, removes evidence, erases a stop condition, hides an unknown, or upgrades DELAY/BLOCK to PASS, it is not decision-equivalent.

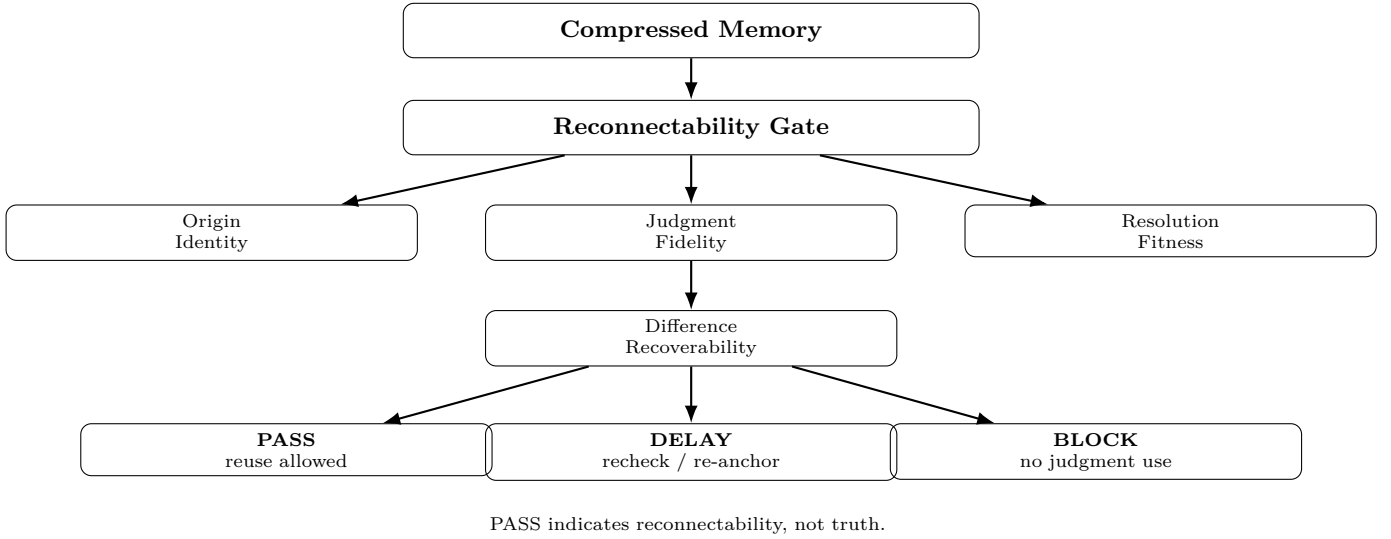


Fig. 2. Reconnectability Gate. Compressed memory is evaluated through origin identity, judgment fidelity, resolution fitness, and difference recoverability. PASS indicates reconnectability, not truth; DELAY requests additional reconnection; BLOCK prevents use as a judgment ground.

VI. RECONNECTABILITY GATE

A compressed memory should not be reused directly. Even if it is readable and linked to a source, it may still be unsafe as judgment material. V11 therefore introduces the *Reconnectability Gate*: a control layer that determines whether a compressed memory may be reused, delayed for re-anchoring, or blocked from judgment use.

The gate does not determine truth. It determines reconnectability.

The gate has three outputs:

- **PASS**: the memory is reconnectable enough for reuse as judgment material;
- **DELAY**: the memory is partially reconnectable but requires additional reconnection;
- **BLOCK**: the memory must not be used as a judgment ground.

PASS means that the compressed memory preserves enough structure for a future evaluator to reconnect, verify, reuse, revise, or stop it under the declared conditions. A PASS memory should preserve the relevant residue or decision, source or evidence anchor, As-of context, scope, stop or recheck condition, unresolved delta, re-entry path, and compression granularity.

PASS is not a truth guarantee. It indicates reconnectability, not correctness.

DELAY means that some usable structure remains, but not enough for direct judgment reuse. Typical DELAY cases include incomplete evidence anchors, unclear As-of conditions, underspecified scope, missing stop conditions, source references that do not identify the relevant part, or compression that appears too coarse for the decision impact. DELAY is not failure. It is a request for additional reconnection, such as reopening the source, restoring As-of context, lowering compression ratio, retrieving evidence, or requesting human confirmation. DELAY must not become a permanent holding state. If DELAY accumulates across repeated reuse attempts, the system should either re-anchor the source, reduce reuse authority, request human review, or close the residue as non-judgment material.

BLOCK means that the compressed memory is not safe to use as a judgment ground. Typical BLOCK cases include unknown origin, missing evidence anchor, contradiction with the source, loss of critical structure, disappearance of a stop condition, or compression that turns DELAY/BLOCK into apparent PASS. BLOCK does not necessarily require deletion. It means exclusion from judgment use until repaired, re-anchored, archived, or reconstructed.

The gate evaluates four criteria.

A. Origin Identity

Origin Identity asks whether the compressed memory can be traced back to its source. This may include source reference, timestamp, version, commit, artifact ID, hash, retrieval path, or author/system boundary. Origin Identity is necessary because a compressed memory without origin cannot be verified or reopened.

However, origin identity is only the entrance to reconnectability. It does not prove judgment fidelity.

B. Judgment Fidelity

Judgment Fidelity asks whether the compressed memory preserves the structures required to recover the same or safer decision state. This includes judgment-critical queries, evidence anchors, stop conditions, recheck or invalidation triggers, unresolved deltas, scope boundaries, As-of context, capability ceiling, and PASS/DELAY/BLOCK state.

If judgment fidelity is uncertain, the gate should downshift to DELAY rather than upgrade to PASS.

C. Resolution Fitness

Resolution Fitness asks whether the compression granularity is appropriate for the memory’s impact, reversibility, evidence dependence, and future re-entry cost. Low-impact reversible material may be stored as coarse residue. High-impact or irreversible material may require detailed residue, stronger evidence anchors, protected archive, or human review.

Resolution Fitness prevents two opposite failures: over-compression, where important structure is lost, and over-retention, where memory remains too heavy to serve the purpose of forgetting.

D. Difference Recoverability

Difference Recoverability asks whether a future mismatch can be identified, explained, and used as a delta. A compressed memory does not need to reproduce the original perfectly. But if a future evaluator reaches a different conclusion, the system should be able to trace whether the difference came from changed evidence, changed scope, omitted assumptions, shifted capability ceiling, new objectives, compression loss, or a genuine external update.

Imperfect recovery is acceptable. Untraceable drift is not.

The gate output is determined conservatively:

- If origin identity, judgment fidelity, resolution fitness, and difference recoverability are sufficient, output PASS.
- If one or more elements are incomplete but repairable, output DELAY.
- If origin, evidence, critical structure, or stop conditions are lost in a way that creates false judgment authority, output BLOCK.

When judgment fidelity cannot be confirmed, the gate outputs DELAY or BLOCK regardless of other criteria. The remaining three criteria are individually necessary but not sufficient.

The gate can itself be misused. Common misuse patterns include reading PASS as truth, treating DELAY as failure, treating provenance as sufficient, trusting readability, blocking all imperfect memories, or upgrading through compression. The gate is designed to prevent compressed memory from gaining false authority. It should not become a decorative checklist that automatically blesses reuse.

The minimal rule is:

A compressed memory may be reused only when it remains reconnectable through origin identity, judgment fidelity, resolution fitness, and difference recoverability.

If reconnectability is incomplete but repairable, the memory should be delayed. If reconnectability is broken in a way that creates unsafe judgment authority, the memory should be blocked.

VII. RECOMPRESSION STOP AND EVALUATION BUDGET

Reconnectable Forgetting does not imply indefinite recompression. Compression reduces active context burden, but each additional compression step can also weaken evidence access, blur stop conditions, smooth unknowns, or make future differences harder to recover. V11 therefore treats recompression as a bounded operation rather than an optimization target.

The objective is not the shortest possible memory. The objective is the lightest memory that still preserves future re-entry.

A recompression process should stop when any of the following conditions holds:

- further compression no longer improves future restartability;
- judgment fidelity begins to decline;
- evidence anchors, As-of conditions, or re-entry paths become weaker;
- stop, recheck, or invalidation conditions become harder to recover;
- differences become less traceable;
- readability improves while judgment structure becomes thinner;
- the cost of further evaluation exceeds the expected memory saving.

This rule prevents a common failure: treating a cleaner summary as a safer memory. A residue may become easier to read while becoming less reconnectable. In V11, elegance is not the criterion. Reconnectability is.

Evaluation must also be budgeted. A compressed memory may require checking against its source, evidence anchor, As-of condition, stop condition, and declared judgment-critical queries. These checks are useful, but they are not free. If every low-impact residue receives full verification, the system spends more effort maintaining memory than it saves by forgetting. If high-impact residue receives only lightweight checking, the system preserves efficiency while losing safety.

V11 therefore assigns evaluation effort according to expected misuse risk.

Evaluation budgeting is not only about how much checking to spend. It also defines how much judgment equivalence each class of memory must preserve.

V11 does not require all residues to target full judgment equivalence. The required fidelity scales with impact and reversibility. Low-impact discovery material may tolerate coarse compression because it has no judgment authority before recheck. Ordinary operational residue requires enough fidelity to support safe re-entry and DELAY/PASS decisions. High-impact or irreversible memory requires near-complete preservation of judgment-critical structure, or it should remain protected at higher resolution.

The gate is not intended to reopen the full source for every residue. Full source replay is reserved for high-impact, irreversible, or replay-failing cases; lower-impact residues may remain lightweight until they are promoted to judgment use.

The evaluation budget should increase when a memory is high-impact, difficult to reverse, evidence-dependent, tied to responsibility, connected to a stop condition, or likely to be reused as judgment material. It may remain lightweight when the residue is low-impact, reversible, easily rechecked, or stored only as discovery material.

The minimal rule is:

Continue recompression only while it improves future re-entry without reducing judgment fidelity below the required threshold. Evaluate compressed memory only to the degree justified by its expected impact and misuse risk.

If further compression makes the memory lighter but less reconnectable, stop. If further checking costs more than the residue is worth, downshift to DELAY, reduce reuse authority, or close the residue as non-judgment material. If the memory is critical, preserve the anchor rather than pretending that a clean residue is enough.

VIII. DISCOVERY LANE FOR LOW-IMPACT RESIDUE

Reconnectable Forgetting must not preserve only high-impact memory. Some residues are low impact at the time they appear, but later become important because they re-emerge, connect across domains, or resist ordinary compression. If such residues are deleted too early, the system may lose future discovery material. If they are preserved too heavily, they become another source of context burden.

V11 therefore separates two lanes:

- **Impact Lane:** memory whose loss could damage judgment, safety, responsibility, evidence, or auditability.
- **Discovery Lane:** low-impact residue that has no present judgment authority but may become future structure.

The Impact Lane uses hard admission. Evidence anchors, stop conditions, responsibility boundaries, irreversible decisions, capability ceilings, dependencies, and recheck triggers must remain reconnectable because losing them may damage future judgment.

The Discovery Lane uses soft admission with delayed promotion. A residue may enter a temporary interest buffer when it shows a weak signal of future relevance, such as re-emergence across contexts, unresolved pull after compression, a plausible cross-domain link, compression resistance, or a clear future re-entry trigger. A candidate is not yet a reusable judgment. It is only a lightweight observation that may deserve later review.

Promotion into the Discovery Lane requires a stronger condition: at least two weak signals, persistence after recheck, a clear re-entry trigger, or connection to an active research, design, or decision frame. This prevents subjective interest from becoming evidence and prevents the Discovery Lane from becoming a second active context.

A minimal Discovery Lane record may contain:

```
{
  "residue": "brief unresolved structure",
  "interest_reason": "re_emergence | unresolved_pull | cross_domain_link |
                    compression_resistance | future_reentry",
  "as_of": "timestamp or version",
  "source_ref": "file, log, thread, or artifact",
  "reentry_trigger": "when to revisit",
  "status": "discovery_candidate"
}
```

This form is intentionally small. It preserves possible future structure without granting present authority. If a Discovery Lane residue is later used for judgment, it must return to the Reconnectability Gate. It does not become PASS merely because it was preserved.

The minimal rule is:

High-impact memory is preserved because losing it would be dangerous. Low-impact recurring residue may be preserved because it may become future structure. Discovery Lane memory has no judgment authority until it passes reconnectability and judgment-fidelity checks.

IX. LINE SELF-RECURSION

Reconnectable Forgetting changes how self-recursion is understood in long-horizon systems. A long-running agent does not need to preserve one uninterrupted active context forever. It does not need the same session, model, prompt, or memory state to continue without rupture.

The important condition is different:

When one point closes, can the next point reconnect to the line?

V11 calls this condition *line self-recursion*.

Point recursion assumes that the same active system carries its own past forward. The agent remembers, updates, reasons, and continues inside one persistent context. This assumption is fragile. Sessions end, models change, files move, histories are summarized, tools are updated, human goals shift, and future agents may operate under different capability ceilings.

If self-recursion is defined as uninterrupted continuity inside one point, then every boundary becomes a failure. V11 defines continuity differently: a point may close if it leaves behind enough structure for the next point to reconnect. This structure includes compressed residue, evidence anchors, As-of conditions, stop and recheck conditions, unresolved deltas, scope boundaries, and re-entry paths.

The point does not have to remain alive. The line has to remain reconnectable.

This also reduces self-referential premise corrosion. In single-context recursion, the system repeatedly reasons from its own prior compressions of the past. Older assumptions may remain active after they expire, local summaries may become treated as original premises, and local decisions may harden into global rules. Line self-recursion lowers this risk by allowing contexts to close while preserving reconnectable residue.

Line self-recursion also permits distributed continuation. The future evaluator may be a later session, a different model, a human operator, an audit process, or another system continuing the same project. This does not automatically break continuity. The same model can fail to continue if it loses reconnectability; a different model can continue if it receives sufficient re-entry structure.

V11 therefore separates substrate identity from operational continuity. Continuity depends not on sameness of model or context, but on whether the future evaluator can recover the relevant judgment environment.

This is the boundary with broader self-evolution theory. V11 does not claim that memory economy alone produces self-evolution. It claims that a self-recursive system cannot remain long-horizon if every point must carry the full burden of the line. Without reconnectable memory economy, self-recursion becomes overloaded, corroded, or non-restartable.

The minimal rule is:

Self-recursion does not require one point to carry everything. It requires each point to leave enough structure for the next point to reconnect.

A point may close. A context may end. A model may change. If the line remains reconnectable, recursion can continue. If the line loses its residue, evidence, conditions, deltas, and re-entry paths, apparent continuity becomes only a memory illusion.

X. BOUNDARY TO V12: FROM RECONNECTABLE MEMORY TO RESTARTABLE COMPLETION

V11 defines a memory condition. V12 defines a closure condition.

The distinction is important. Reconnectable Forgetting asks whether past context can leave active memory without destroying the future system's ability to return to the relevant evidence, scope, As-of condition, unresolved delta, stop condition, and re-entry path. Completion Integrity asks whether an update or task can be closed without preventing the future self from restarting, verifying, stopping, correcting, or re-anchoring the work [3].

In short:

V11 governs what may be forgotten. V12 governs what may be closed.

The two frameworks are connected, but they should not be merged. If V11 is expanded into a general theory of completion, its core contribution becomes diluted. The task of V11 is narrower: to define the memory economy that makes long-horizon continuation possible without total recall.

A compressed memory can pass V11’s Reconnectability Gate while the broader task still fails V12’s Completion Gate. For example, a residue may preserve its source, As-of condition, stop rule, and re-entry path, yet the current task may still have unresolved implementation work, missing handoff instructions, or an unverified closure claim. In that case, the memory is reconnectable, but the work is not complete.

Conversely, a task cannot be safely completed if its supporting memory has become non-reconnectable. If the future self cannot recover the evidence, scope, open delta, or stop condition behind the closure, then completion becomes fragile even if the current artifact appears finished.

The boundary is therefore:

- **V11:** Can the future system reconnect to the memory?
- **V12:** Can the future system restart the work from the closure?

This paper stops at the first question. It does not define Completion Integrity, False Completion, or a Completion Gate. Those belong to V12. V11 only supplies the memory-side prerequisite: forgetting is allowed only when the future self retains a path back.

The minimal rule is:

Reconnectable memory is a precondition for restartable completion, but it is not completion itself.

Thus, V11 hands forward a bounded claim. Long-horizon agents do not need perfect memory. They need memory that can become light enough to continue while remaining structured enough for future closure, audit, and re-entry.

XI. DISCUSSION

V11 should be read as a memory-governance framework, not as a claim that compressed memory is automatically safe.

The central distinction is between *origin identity* and *judgment fidelity*. A Provenance-Key Layer can show where a residue came from, when it was created, and how to reopen its source. This is necessary for reconnectability. However, it does not prove that the compressed residue preserved the judgment-critical structure of the original memory.

A memory can have valid provenance and still fail judgment fidelity. It may point to the correct source while omitting the stop condition. It may preserve the correct timestamp while losing the capability ceiling. It may preserve the correct file reference while smoothing an unresolved assumption into a conclusion.

Therefore, the governance principle of V11 is:

Memory reuse requires governance before reuse.

This means that a compressed memory should not be reused merely because it exists, is readable, or has a source reference. Before reuse, the system must ask whether the memory remains reconnectable through origin identity, judgment fidelity, resolution fitness, and difference recoverability.

This position separates V11 from ordinary long-term memory systems. Many memory systems optimize for storage, retrieval, semantic similarity, or summarization quality. V11 instead asks whether a future evaluator can safely use the compressed memory as judgment material. The object of evaluation is not recall alone, but condition-bound reuse.

This also separates V11 from cryptographic claims. The Provenance-Key Layer is not a cryptographic protocol. It preserves origin and re-entry pointers without expanding raw content into active context. V11 does not claim to implement a zero-knowledge system, prove privacy guarantees, or solve all identity and access-control problems.

The framework is also not a general theory of self-evolution. Reconnectable Forgetting may be necessary for long-horizon self-recursion, but memory economy alone does not produce self-evolution. V11 only defines how past context can leave active memory while preserving enough structure for future re-entry.

The main practical implication is conservative reuse. If a compressed memory cannot answer the declared judgment-critical queries, it should not be treated as PASS. If the missing structure is repairable, it should become DELAY. If origin, evidence, stop condition, or critical scope has been lost, it should become BLOCK for judgment use.

This conservative rule prevents compressed memory from gaining false authority. A fluent residue should not become a hidden premise. A readable summary should not become a permanent permission. A valid source pointer should not be mistaken for preserved judgment.

In this sense, V11 treats memory as an operational object. A memory is not useful merely because it can be retrieved. It is useful when the future system can determine how it may be reused, when it must be rechecked, and when it must not support judgment.

The discussion can be summarized as follows:

Reconnectable Forgetting is not a storage policy. It is a governance condition for when forgotten context may safely re-enter judgment.

XII. FALSIFIER AND SCOPE BOUNDARY

V11 is an operational claim, not a metaphor about memory.

The framework is weakened or falsified if long-horizon agentic AI systems can consistently maintain cost efficiency, judgment quality, auditability, and safe reuse by either retaining full-resolution history or using ordinary summarization alone, without reconnectable compression, provenance keys, judgment-fidelity checks, or reconnectability gates.

In minimal form:

If total recall or ordinary summarization consistently outperforms reconnectable compression under long-horizon reuse, V11 loses its reason to exist. Operational studies may approximate future re-entry stability through replay success rate, evidence-anchor recovery rate, DELAY/BLOCK inflation rate, and the frequency of unsafe PASS upgrades after compression.

For this falsifier to hold, the alternative must outperform V11 across three axes simultaneously: cost efficiency, decision quality, and future re-entry stability. Lower cost alone is not sufficient if future re-entry becomes unstable. Higher apparent decision quality is not sufficient if the supporting evidence, scope, or stop conditions become non-reconnectable. V11 is weakened only when simpler memory strategies preserve all three more effectively under comparable long-horizon conditions.

This falsifier matters because Reconnectable Forgetting is not valuable merely because it sounds elegant. It is valuable only if long-horizon agents face a real tradeoff between active context burden and future judgment fidelity. If that tradeoff disappears, or if simpler memory strategies solve it without loss, the proposed framework becomes unnecessary.

The framework can also fail at the implementation level. A system claiming Reconnectable Forgetting fails if compressed memories routinely lose evidence anchors, erase As-of conditions, hide unknowns, weaken stop conditions, or upgrade DELAY/BLOCK states into apparent PASS. It also fails if the Provenance-Key Layer preserves origin identity but the system treats that origin identity as sufficient proof of judgment fidelity.

V11 therefore makes a conservative prediction: future long-horizon agents will need memory systems that distinguish storage, summarization, provenance, judgment fidelity, and condition-bound reuse. If practical systems do not require these distinctions, the framework is over-specified.

The scope of V11 is intentionally narrow.

It does not define a complete long-term memory architecture. It does not prescribe a vector database, retrieval engine, embedding method, compression algorithm, access-control system, or agent framework. It does not implement a cryptographic zero-knowledge protocol. It does not claim empirical validation of all proposed gates. It does not claim that raw data should always be deleted, nor that all memory should be compressed.

Some material should remain protected at higher resolution. This includes critical evidence anchors, irreversible decisions, responsibility boundaries, safety-relevant logs, source artifacts required for audit, and records whose compression would make later verification impossible. Reconnectability also depends on the durability of external anchors. Files, APIs, permissions, repositories, and links may move or decay over long horizons. V11 therefore treats provenance keys as re-entry commitments, not guarantees of permanent access. High-impact memories should preserve redundant anchors, hashes, archived artifacts, or human-readable fallback records when source loss would make future judgment unrecoverable.

V11 also does not define Completion Integrity, self-evolution, or release governance. Those belong to adjacent layers in the Decision-OS lineage. V11 only defines the memory-economy condition required for long-horizon continuation:

A system may reduce active memory only when the future evaluator retains a structured path back to what matters.

Thus the contribution is bounded but specific. V11 defines the minimal conditions under which forgetting can reduce context burden without destroying future re-entry.

XIII. CONCLUSION

Long-horizon agentic AI cannot rely on either total memory or ordinary summarization. Total memory preserves detail but increases active burden, retrieval cost, and premise-corrosion risk. Ordinary summarization reduces length but may remove evidence, stop conditions, unresolved deltas, As-of boundaries, and re-entry paths.

V11 defines a third position: *Reconnectable Forgetting*.

Forgetting, in this framework, is not deletion. It is a controlled compression operation that allows past context to leave active memory while preserving the structures required for future re-entry. A memory is safely forgotten only when the future evaluator can still reconnect to the relevant claim, evidence anchor, As-of condition, scope, stop or recheck condition, unresolved delta, and re-entry path.

The paper introduced four core components.

First, the three-layer memory architecture separates Active Context, Compressed Residue, and Provenance-Key functions. Active context supports current high-resolution work. Compressed residue preserves reusable judgment structure. The Provenance-Key Layer preserves origin identity and re-entry without expanding raw history back into active context.

Second, Decision-Equivalent Compression distinguishes meaning preservation from judgment preservation. A compressed memory is not safe merely because it is fluent or semantically close. It is safe only if it preserves the judgment-critical structure required to recover the same or safer decision state.

Third, the Reconnectability Gate evaluates compressed memory through origin identity, judgment fidelity, resolution fitness, and difference recoverability. PASS permits reuse as judgment material, DELAY requires recheck or re-anchoring, and BLOCK prevents use as a judgment ground.

Fourth, the framework defines bounded controls for recompression, evaluation budget, discovery residue, and line self-recursion. These controls prevent memory economy from becoming over-compression, endless inspection, or false continuity.

The main limitation is that V11 is a conceptual and operational framework, not a complete implementation. It does not specify a vector database, retrieval architecture, cryptographic zero-knowledge system, privacy policy, or empirical validation protocol. It also does not claim that all raw memory should be deleted. Some material must remain protected, archived, or human-reviewed.

The contribution of V11 is narrower and more basic: it defines the minimal condition under which forgetting can reduce context burden without destroying future re-entry.

In one line:

Future requires forgetting, but forgetting must remain reconnectable.

A long-horizon agent does not need to remember everything. It must know what can leave active context, what must remain anchored, what cannot be compressed, and when a future self must reopen the source. If those conditions are preserved, memory becomes light enough to continue while remaining structured enough to reconnect.

DISCLOSURE AND USE OF AI TOOLS

This paper was developed by the author with the assistance of AI tools for drafting, restructuring, translation support, terminology refinement, and consistency checking.

All conceptual claims, framework decisions, final wording choices, publication decisions, and responsibility for the content remain with the author.

AI tools were used as assistants for iteration and editorial support. They were not treated as decision owners, empirical validators, or independent authorities. No AI system is credited as an author of this paper.

The framework proposed in this paper is conceptual and operational. It does not claim empirical validation, deployment testing, or implementation as a complete memory architecture. The proposed gates, layers, and criteria should be read as design primitives for long-horizon agentic AI systems rather than as experimentally verified components.

The author also notes that this paper is part of the broader Decision-OS research sequence. Earlier Decision-OS works introduced related concepts such as evidence anchoring, As-of fixation, impact-weighted release, survival-bounded planning, and completion integrity. V11 focuses specifically on reconnectable forgetting as a memory-economy condition for future re-entry.

REFERENCES

- [1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [2] L. Moreau and P. Missier, “Prov-dm: The prov data model,” World Wide Web Consortium, W3C Recommendation, Apr. 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [3] S. Nagata, “Decision-os v12: Completion integrity. doi: 10.5281/zenodo.20102241,” 2026, decision-OS research sequence.
- [4] —, “Decision-os v5: Siriusa – zero-knowledge confirmation layer for the protection of life,” 2026, decision-OS research sequence.
- [5] —, “Decision-os v6: Canonical memory architecture for self-recursive intelligence,” 2026, decision-OS research sequence.
- [6] —, “Decision-os v7: Aspire intelligence for agi,” 2026, decision-OS research sequence.
- [7] —, “Decision-os v8: Time-tube control for self-safe agi,” 2026, decision-OS research sequence.
- [8] —, “Decision-os v9.1: Impact-weighted release,” 2026, decision-OS research sequence.
- [9] —, “Decision-os v10 note: Survival-bounded planning,” 2026, decision-OS research sequence.
- [10] —, “Decision constitution,” 2026, decision-OS constitutional document.